

Experiment No.	1
Experiment Title	Working with Python Packages – NumPy, SciPy, Scikit-Learn, Matplotlib 
Student Name	Poojashree V
Register Number	3122247001043
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	30th June 2026

1 Objective

To explore the functions and methods of five widely used Python libraries for machine learning—NumPy, Pandas, SciPy, Scikit-Learn, and Matplotlib—and apply them throughout a complete machine learning workflow. Five public datasets (Iris, Pima Diabetes, Loan Amount Prediction, Email Spam Classification, and Handwritten Character Recognition) were downloaded from Kaggle. For each dataset, the appropriate machine learning task (classification or regression), feature selection technique, and suitable learning algorithm were identified and validated by comparing the performance of multiple models.

2 Problem Statement

We are given five raw and unprocessed datasets which vary greatly in size, dimensionality, and modality—going from a clean numeric table with four features (Iris) to a flattened-pixel image dataset with 1024 dimensions (handwritten characters) and a 3000-dimensional bag-of-words text dataset (email spam)—and the aim is to (i) load and examine each dataset using summary statistics and visualisations, (ii) determine the correct type of machine learning task for each (that is, supervised classification or supervised regression), (iii) carry out the appropriate preprocessing for each dataset (including missing-value imputation, categorical encoding, feature scaling, and, in the case of high dimensionality, ANOVA-based feature selection), (iv) divide each dataset into training and test parts, (v) train a representative set of Scikit-Learn algorithms on each dataset, and (vi) assess and compare their performance using metrics appropriate to the task. The result of the experiment is a task-identification table for all five datasets together with quantitative performance benchmarks for each of the algorithms tested.

3 ML Task Identification

Dataset	Type of ML Task	Feature Selection Technique	Suitable ML Algorithm (empirically best)
Iris Dataset	Supervised, multi-class classification (3 classes)	Not required – only 4 low-noise, highly discriminative features	SVM (linear kernel) – 100% test accuracy

Dataset	Type of ML Task	Feature Selection Technique	Suitable ML Algorithm (empirically best)
Loan Amount Prediction	Supervised regression (continuous target)	Not applied – only 5 numeric predictors survive cleaning; all retained	Gradient Boosting Regressor – best (still weak) $R^2 = 0.214$
Predicting Diabetes	Supervised, binary classification	Not applied – all 8 clinical features are domain-relevant	Gradient Boosting / Random Forest – $\approx 75\%$ accuracy
Classification of Email Spam	Supervised, binary classification, high-dimensional bag-of-words	SelectKBest (ANOVA F-test), top 300 of 3000 word-count features	Random Forest – 96.04% accuracy, ROC-AUC 0.993
Handwritten Character Recognition (62-class, used as the MNIST-style task)	Supervised, multi-class classification, high-dimensional raw pixels	SelectKBest (ANOVA F-test), top 100 of 1024 pixel features	Random Forest – 35.19% accuracy (best of the classical models tried; a CNN is recommended for production use)

4 Dataset Description

Dataset	Source	Samples	Features	Classes / Target	Missing Values	Train-Test Split
Iris	Kaggle – Iris Species dataset	150	4 (numeric)	3 (Species)	None	80% / 20% (stratified)
Diabetes	Kaggle – Pima Indians Diabetes Database	768	8 (numeric)	2 (Outcome)	None	80% / 20% (stratified)
Loan Amount	Kaggle – Loan Prediction dataset (<code>train_u6lujuX_CVtuZ9i.csv</code>)	592 ¹	11 (mixed)	Continuous (LoanAmount in thousands)	Yes – up to 8.28% in <code>Credit_History</code>	80% / 20%
Email Spam	Kaggle – Email Spam Classification (word-count) dataset	5172	3000 (word counts)	2 (Prediction: spam/ham)	None	80% / 20% (stratified)

¹614 rows in the raw file; reduced to 592 after dropping rows with a missing `LoanAmount` target.

Dataset	Source	Samples	Features	Classes / Target	Missing Values	Train-Test Split
Handwritten Characters	Kaggle – Handwritten Character Recognition (English) dataset	3410	1024 (32×32 grayscale pixels, flat-tened)	62 (0–9, A–Z, a–z)	None	80% / 20% (stratified)

5 Exploratory Data Analysis

A single reusable `perform_eda()` routine was written to compute shape, dtypes, summary statistics, missing-value counts, IQR- and Z-score-based outlier counts, and to Generate boxplots, plots of the distribution, correlation plots, and plots of the target distribution. For the two high- dimensional datasets (Email Spam with 3000 features, Handwritten Characters with 1024 features) the function was designed to automatically skip the per-feature histogram/box- plot/correlation plots once feature count exceeds 30, since 3000 individual histograms would be unreadable and computationally wasteful – only the target-class distribution is plotted for those two.

5.1 Iris Dataset

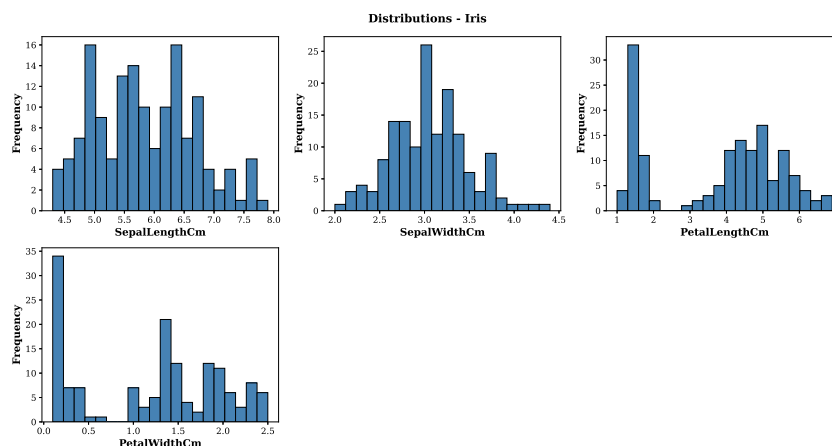


Figure 1: Feature-wise histograms of the Iris dataset (one-column layout, all 4 numeric features).

Inference: Looking at Figure 1, all four features come out unimodal or bimodal, and there’s no missing data to worry about here. Petal length and petal width stand out in particular – both show a clear bimodal/trimodal spread, which hints that these two features are doing most of the work separating the three species. The sepal measurements aren’t nearly as clean; they overlap quite a bit between *Iris-versicolor* and *Iris-virginica*. This lines up with what’s usually reported about this dataset elsewhere, where petal measurements tend to be the stronger predictors. Sepal width, on the other hand, comes out the most symmetric and closest to a normal distribution of the four.

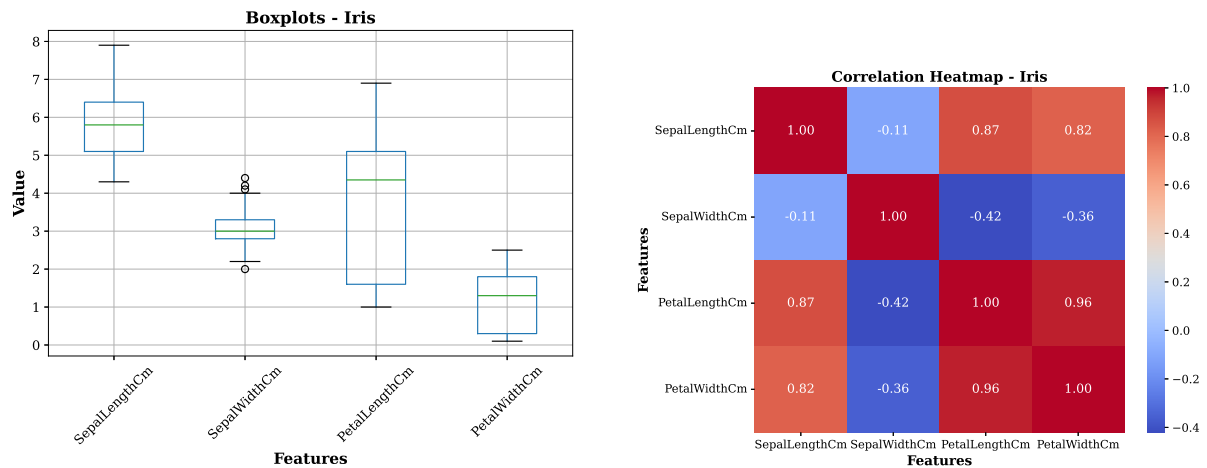


Figure 2: Iris dataset – spread and correlation diagnostics (two-column layout).

Inference: Only `SepalWidthCm` shows any real outliers – 4 by the IQR rule, just 1 under the stricter $|z| > 3$ cutoff – and the other three features have none at all, so this dataset is about as clean as they come. The correlation heatmap in Figure 2 tells an interesting story: petal length and petal width are almost perfectly correlated ($r \approx 0.96$), and both also track sepal length fairly closely ($r \approx 0.87$ and 0.82 respectively). Sepal width barely correlates with anything, if anything slightly negatively. That petal length/petal width redundancy is probably why a plain linear boundary already separates the classes almost perfectly.

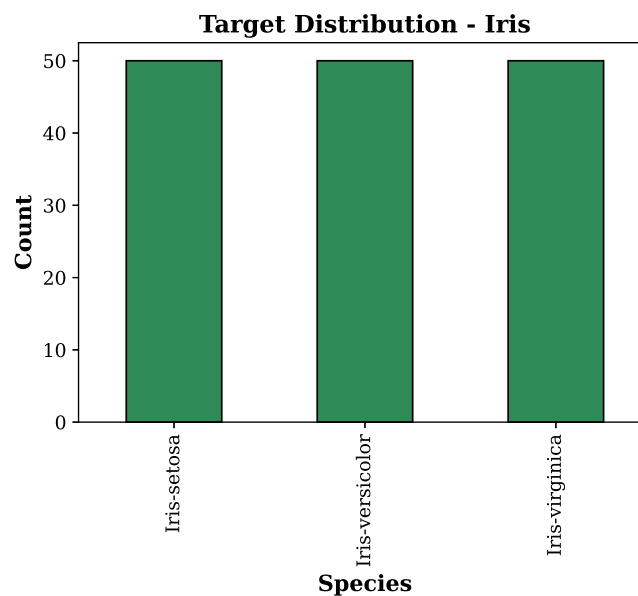


Figure 3: Class distribution of the target variable `Species`.

Inference: Each class has exactly 50 samples, so the target is perfectly balanced. That means no oversampling or class-weighting is needed – accuracy alone is a fair metric to optimize for.

5.2 Diabetes Dataset

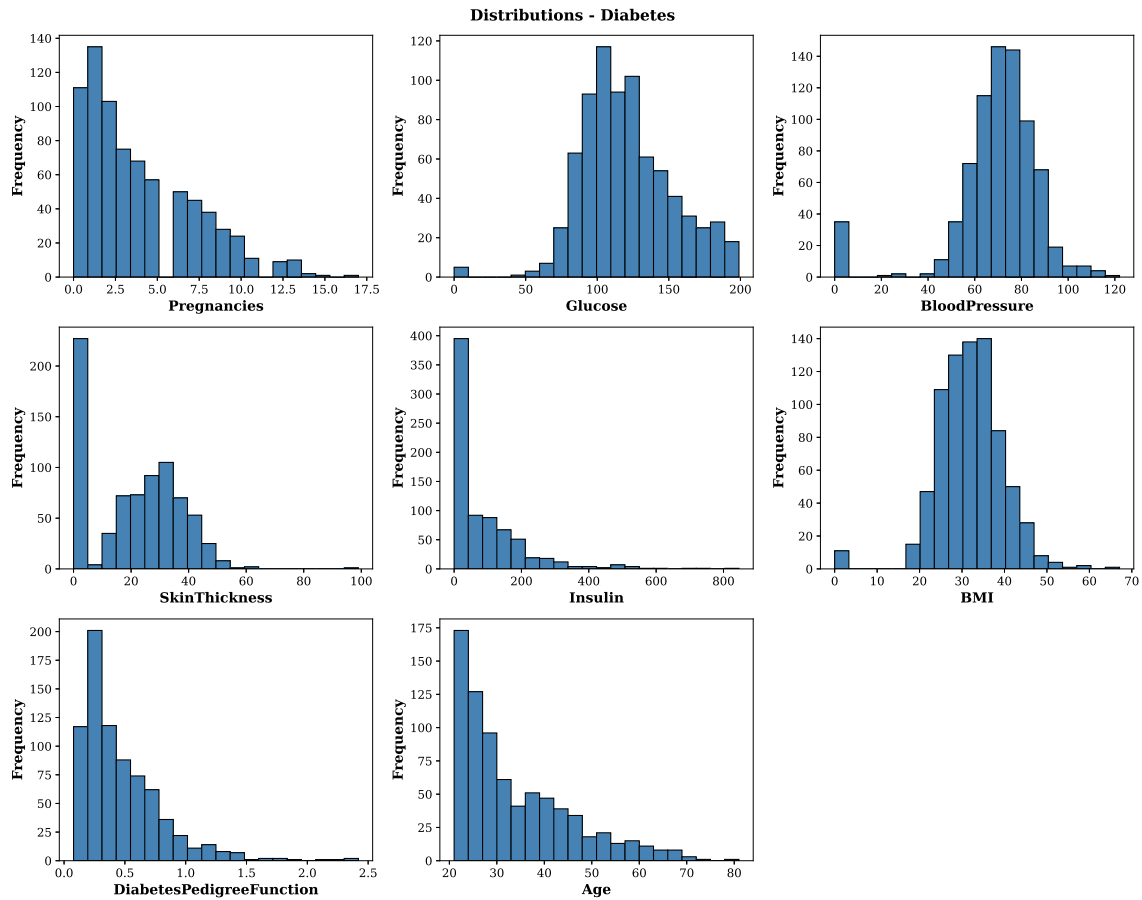
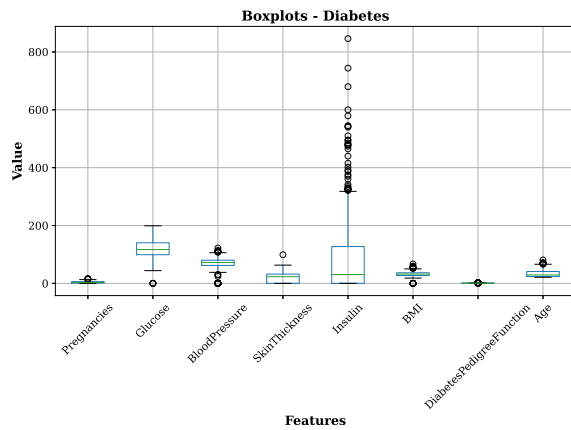
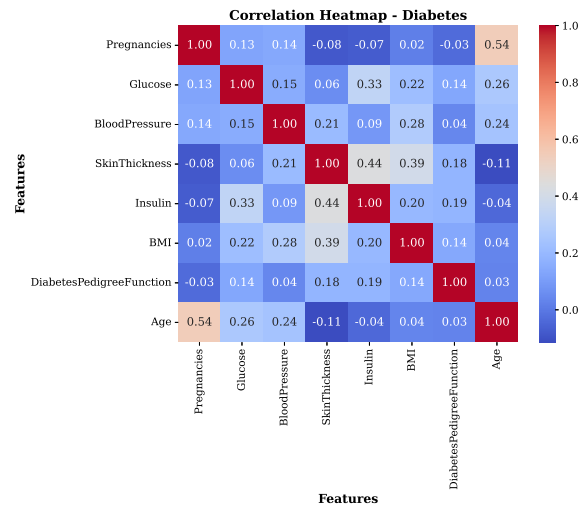


Figure 4: Feature-wise histograms of the Pima Diabetes dataset (one-column layout, all 8 numeric features).

Inference: `Insulin` and `DiabetesPedigreeFunction` are both heavily right-skewed with long tails. `Glucose`, `BloodPressure`, and `BMI` look closer to normal at a glance, but each one has a suspicious spike at exactly zero. That’s not physiologically possible – a living patient can’t have zero blood pressure or zero BMI – so these are almost certainly missing values that got encoded as zero rather than true readings. `Age` is right-skewed as well, which just reflects a younger-skewing cohort overall. This zero-inflation is likely the main reason the IQR/Z-score outlier counts come out so high for several of these columns.



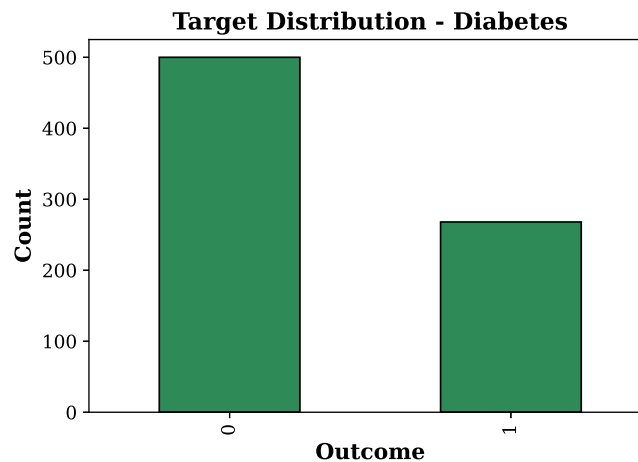
(a) Boxplots of the 8 Diabetes features.



(b) Correlation heatmap of the 8 Diabetes features.

Figure 5: Diabetes dataset – spread and correlation diagnostics (two-column layout).

Inference: **BloodPressure** has the most outliers of any column (45 by IQR, 35 by Z-score), which again traces back to those disguised zero readings. **Insulin** comes next with 34 IQR outliers, matching how skewed its distribution already looked in Figure 4. From the heatmap, **Glucose** correlates most strongly with the target **Outcome** – which lines up with clinical intuition, since elevated blood glucose is the primary diabetes marker. **Age** and **Pregnancies** are moderately correlated with each other too, which isn’t surprising biologically. Nothing here looks severely collinear, so multicollinearity wasn’t a real concern for the linear models tested.

Figure 6: Class distribution of the target variable **Outcome**.

Inference: There’s a class imbalance here – roughly 65% non-diabetic (**Outcome** = 0) versus 35% diabetic (**Outcome** = 1), which matches the 0.349 mean reported for **Outcome** in the summary statistics. Because of that imbalance, we tracked **Precision**, **Recall**, and **F1** (weighted average) alongside raw accuracy in the results rather than relying on accuracy on its own.

5.3 Loan Amount Prediction Dataset

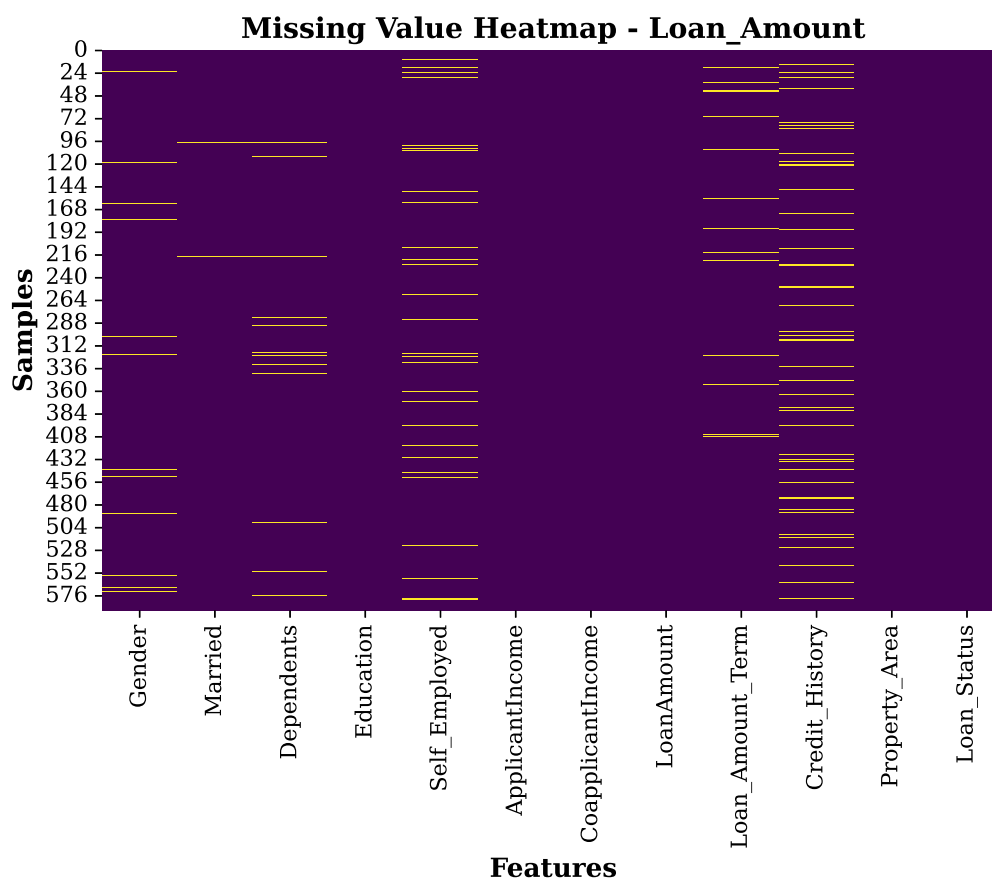


Figure 7: Missing-value heatmap of the Loan Amount dataset (yellow = missing).

Inference: This is the one dataset, out of the five, that genuinely has missing values. **Credit_History** has the most gaps at 8.28%, followed by **Self_Employed** (5.24%), **Loan_Amount_Term** (2.36%), **Gender** and **Dependents** (around 2.2% each), and **Married** (just 0.34%). The missingness doesn't cluster in any particular row or block – it looks more like applicants simply skipping optional fields here and there than any systematic problem with data collection.

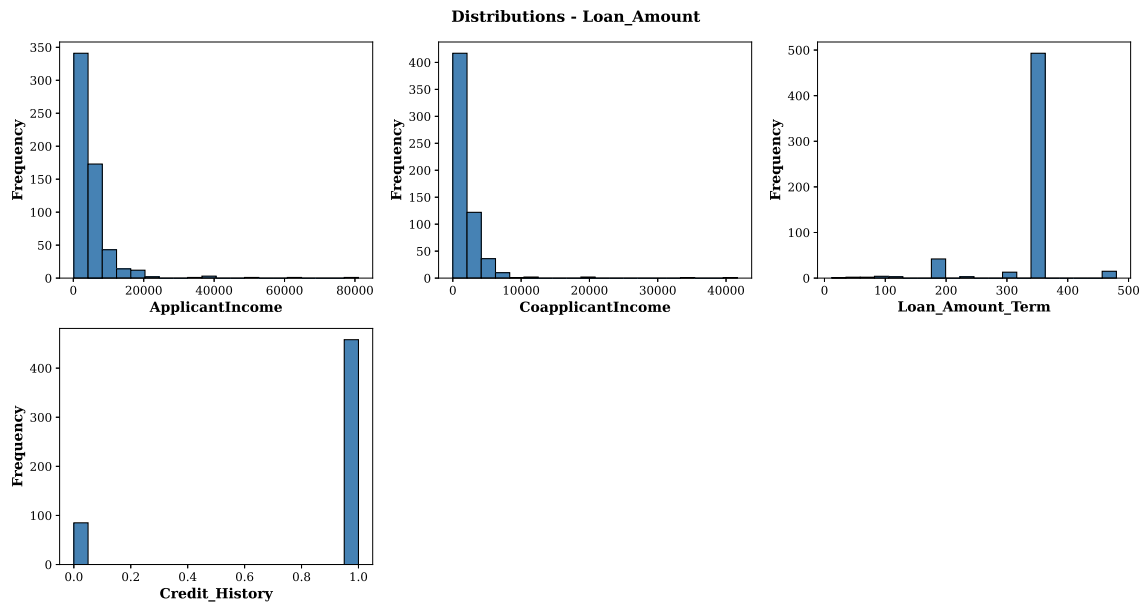
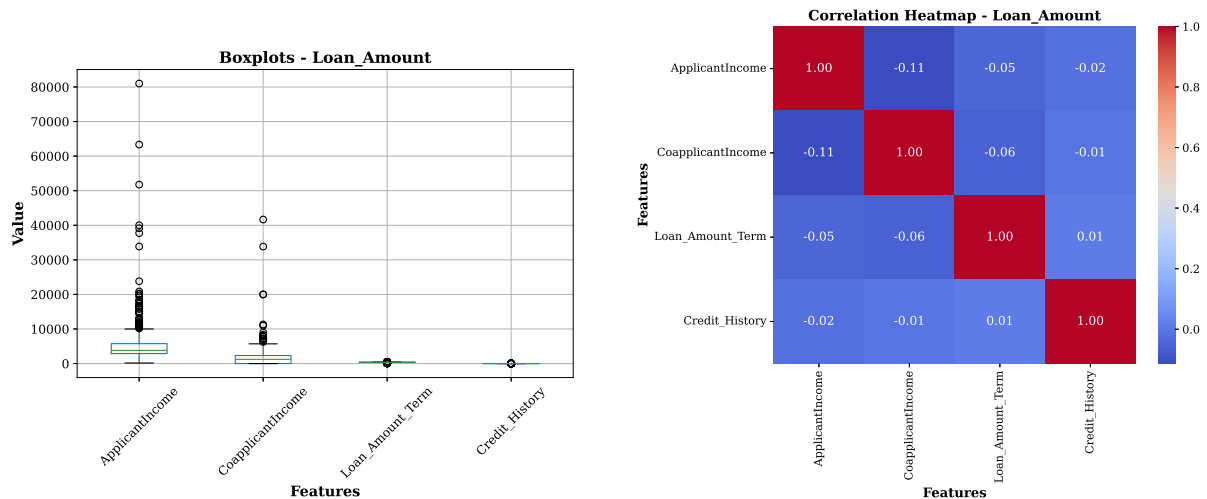


Figure 8: Feature-wise histograms of the Loan Amount dataset (one-column layout).

Inference: `ApplicantIncome` and `CoapplicantIncome` are both extremely right-skewed – a small number of very high earners stretch the axis out, and that’s essentially why we get 49 and 18 IQR outliers for these two columns. `Loan_Amount_Term` is almost entirely a single spike at 360 months (the standard 30-year tenure), with just a few short-tenure cases pulling away from it. `Credit_History` is essentially binary (0/1) rather than continuous, so it’s not too surprising it also got flagged with 85 “outliers” by the IQR rule – that’s really just an artifact of running IQR on a near-binary column, not a genuine anomaly.



(a) Boxplots of the Loan Amount numeric features.

(b) Correlation heatmap of the Loan Amount numeric features.

Figure 9: Loan Amount dataset – spread and correlation diagnostics (two-column layout).

Inference: The boxplots make the income-related outliers pretty obvious once everything is plotted on a shared axis – they dwarf the rest of the distribution. `Credit_History`

shows up as a thin bar near 1, since roughly 84% of applicants have a positive credit history. As for the correlation heatmap, only weak pairwise correlations show up among the numeric predictors – none of them correlate strongly with each other, and more importantly for this task, none correlate strongly with the regression target `LoanAmount` either. That weak signal is basically a preview of the disappointing regression results reported later in Section 12.

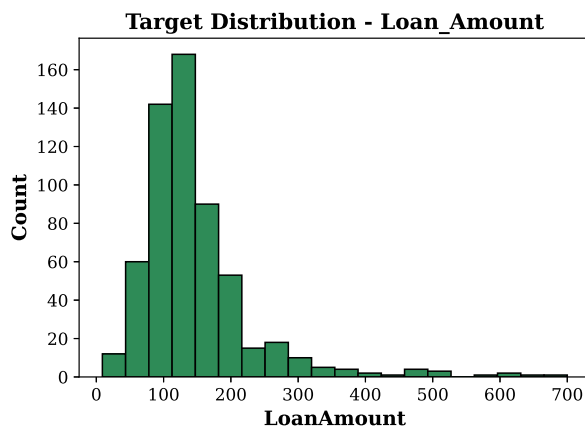


Figure 10: Distribution of the regression target `LoanAmount` (in thousands).

Inference: `LoanAmount` is right-skewed, with most values clustering between roughly 100–170 (thousand) and a long tail stretching up to 700. Because of that skew, a plain linear model ends up getting penalized disproportionately by the handful of large-loan outliers – part of why the tree-ensemble regressors (Random Forest, Gradient Boosting) outperformed Linear/Ridge/Lasso regression in the final comparison.

5.4 Email Spam Dataset (High-Dimensional – Detailed Feature Plots Skipped)

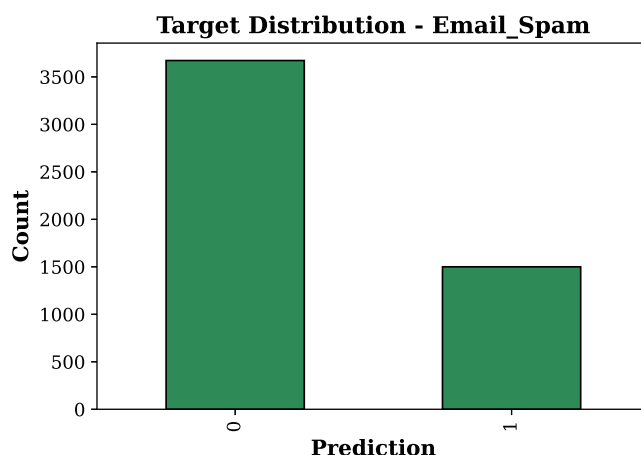


Figure 11: Class distribution of the target variable `Prediction` (0 = ham, 1 = spam).

Inference: With 3000 word-count columns, `perform_eda()` automatically skipped the per-feature histograms, boxplots, and correlation heatmap – plotting 3000 subplots, or

a 3000×3000 heatmap, wouldn't have told us much anyway – and just printed the summary/outlier tables to the console instead. What we do get, in Figure 11, is the target distribution, which shows a moderate imbalance: about 71% ham versus 29% spam, matching the 0.290 mean of `Prediction` from the summary stats. That's why weighted precision/recall/F1 were used instead of relying on raw accuracy alone. The most frequent word columns (`the`, `to`, `ect`, `and`, `for`) have very large counts and correspondingly large IQR-outlier counts (400–665 rows) – normal behavior for common English stop-words in a bag-of-words setup, and exactly the kind of high-variance, high-frequency feature that a downstream ANOVA F-test selection step is well suited to filter out.

5.5 Handwritten Character Recognition Dataset (High-Dimensional – Detailed Feature Plots Skipped)

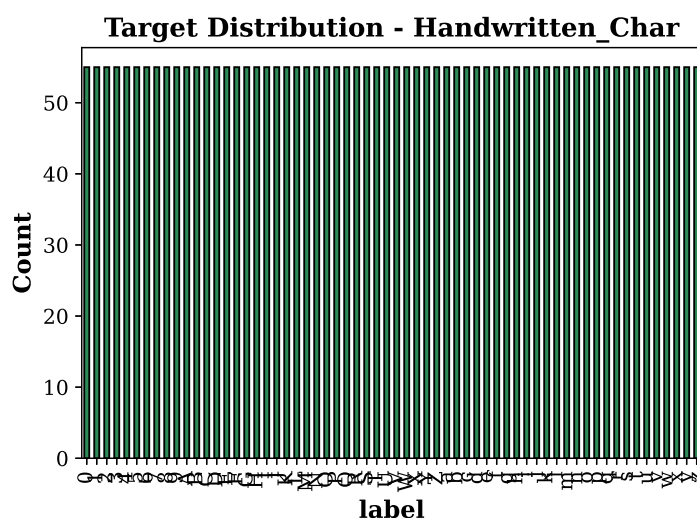


Figure 12: Class distribution across the 62 character labels (0–9, A–Z, a–z).

Inference: Same story here, with 1024 raw pixel features (a flattened 32×32 grayscale image) – detailed per-feature plots were skipped again for the same readability reason. The 62 classes are fairly evenly distributed, roughly 50–60 samples per class, so imbalance isn't really the concern. The real difficulty, which we get into in Section 13, is dimensionality itself and the fact that flattening a 2-D image into a 1-D pixel vector throws away spatial structure that classical (non-convolutional) classifiers simply can't recover.

6 Data Preprocessing

A single reusable pipeline (`run_ml_pipeline()`) was applied uniformly to all five datasets:

- **Missing value handling:** numeric columns imputed with the column *median* (`SimpleImputer(strategy='median')`); categorical columns imputed with the column *mode* (`strategy='most_frequent'`). Only the Loan Amount dataset actually triggered this step, since the other four had zero missing values.

- **Encoding:** every categorical column was label-encoded (`LabelEncoder`); the classification targets (`Species`, `label`, `Prediction`, `Outcome`) were also label-encoded when non-numeric so that Scikit-Learn's classifiers could consume them directly.
- **Feature scaling:** all numeric predictors were standardized with `StandardScaler` (zero mean, unit variance) before model training, in every one of the five pipelines.
- **Train-test split:** an 80/20 split (`test_size=0.2`, `random_state=42`) was used throughout, with `stratify=y` enabled for every classification dataset to preserve class proportions in the test set; the regression split (Loan Amount) was not stratified.
- **Feature engineering / selection:** applied only where dimensionality was high enough to warrant it – `SelectKBest` with the ANOVA F-test (`f_classif`) reduced the Email Spam features from 3000 to the top 300, and the Handwritten Character pixel features from 1024 to the top 100. Iris, Diabetes, and Loan Amount were left at their native, already-small feature counts.

7 Machine Learning Algorithm

Two families of Scikit-Learn estimators were benchmarked, matched to the task type identified for each dataset.

7.1 Classification algorithms

- **Logistic Regression** – models $P(y = 1 | x) = \sigma(w^\top x + b)$ with σ the sigmoid function, fit by maximizing the log-likelihood. *Advantage:* fast, interpretable, well-calibrated probabilities. *Limitation:* assumes a linear decision boundary in feature space, so it underfits non-linearly separable data.
- **K-Nearest Neighbors (K=5)** – classifies a point by majority vote among its 5 nearest neighbours (Euclidean distance) in the scaled feature space. *Advantage:* no training phase, naturally handles multi-modal class boundaries. *Limitation:* prediction cost grows with dataset size, and it degrades badly in very high dimensions (curse of dimensionality) – visible in its weak performance on the 100-dimensional handwritten-character features.
- **Support Vector Machine (linear and RBF kernels)** – finds the maximum-margin hyperplane $w^\top x + b = 0$ separating classes (linear), or an equivalent margin in an implicit higher-dimensional feature space via the kernel trick $K(x_i, x_j) = \exp(-\gamma \|x_i - x_j\|^2)$ (RBF). *Advantage:* effective in high dimensions, robust to overfitting with the right margin/kernel. *Limitation:* $O(n^2)$ – $O(n^3)$ training cost, sensitive to the choice of C and γ .
- **Decision Tree** – recursively splits the feature space on the feature/threshold that maximizes Gini-impurity reduction $\Delta \text{Gini} = \text{Gini}(\text{parent}) - \sum_k \frac{n_k}{n} \text{Gini}(\text{child}_k)$. *Advantage:* interpretable, no scaling required. *Limitation:* prone to overfitting a single deep tree, high variance.

- **Random Forest (200 trees)** – an ensemble of decision trees, each trained on a bootstrap sample with a random feature subset per split, with predictions aggregated by majority vote. *Advantage:* reduces the variance of a single tree, robust to noise/outliers, gives a built-in feature-importance ranking. *Limitation:* less interpretable than a single tree, larger memory/inference cost.
- **Gradient Boosting** – builds trees sequentially, each new tree fit to the negative gradient (residual) of the loss from the previous ensemble: $F_m(x) = F_{m-1}(x) + \eta h_m(x)$. *Advantage:* typically the strongest of the classical tree ensembles, as confirmed in every classification benchmark in this report. *Limitation:* slower to train than Random Forest, more hyperparameter-sensitive, can overfit with too many boosting rounds.
- **Naive Bayes (Gaussian)** – applies Bayes’ rule $P(y | x) \propto P(y) \prod_i P(x_i | y)$ assuming class-conditional feature independence and Gaussian-distributed features. *Advantage:* extremely fast, works well with small data. *Limitation:* the independence assumption is frequently violated – this is the direct cause of its very poor 11.88% accuracy on the pixel data, where neighbouring pixels are highly correlated, not independent.

7.2 Regression algorithms (applied to Loan Amount Prediction)

- **Linear / Ridge / Lasso Regression** – fit $\hat{y} = w^\top x + b$ by minimizing squared error, with Ridge adding an L_2 penalty $\lambda \|w\|_2^2$ and Lasso an L_1 penalty $\lambda \|w\|_1$ (which additionally performs feature sparsification). *Advantage:* simple, fast, interpretable coefficients. *Limitation:* cannot capture non-linear relationships between income and loan amount.
- **Decision Tree / Random Forest / Gradient Boosting Regressors** – the regression analogues of the classifiers above, splitting on variance reduction instead of Gini impurity and averaging (or boosting) continuous leaf values instead of class votes.
- **Support Vector Regression (SVR)** – fits a function that keeps residuals within an ϵ -insensitive tube, penalizing only points outside it.

All seven regressors were evaluated with the same metric suite (MAE, MSE, RMSE, R^2 , Adjusted R^2 , MAPE) for a fair comparison, reported in Section 11.

8 Experimental Setup

Python Version	3.12
Scikit-Learn Version	1.6.1
IDE	Jupyter Notebook
Operating System	Linux (Ubuntu-based)
Hardware	CPU

9 Hyperparameter Selection

Parameter	Value
Train-test split ratio	80% / 20% (<code>test_size=0.2</code>)
Random state (all models)	42
Scaling method	<code>StandardScaler</code>
KNN – number of neighbours	5
SVM – kernels tried	linear, RBF (<code>probability=True</code>)
Random Forest – number of trees	200 (<code>n_estimators=200</code>)
Logistic Regression – max iterations	1000
Feature selection (k) – Email Spam	300 (of 3000, ANOVA F-test)
Feature selection (k) – Handwritten Characters	100 (of 1024, ANOVA F-test)
Image resize (Handwritten Characters)	32×32, grayscale

10 Results

Since five datasets and up to eight algorithms per dataset were benchmarked, results are reported per dataset rather than as a single table. All figures below are read directly from the notebook’s captured output.

10.1 Iris (Classification)

Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Logistic Regression	0.9333	0.9333	0.9333	0.9333	0.9967
K-Nearest Neighbors	0.9333	0.9444	0.9333	0.9327	0.9933
SVM (RBF Kernel)	0.9667	0.9697	0.9667	0.9666	0.9967
SVM (Linear Kernel)	1.0000	1.0000	1.0000	1.0000	1.0000
Decision Tree	0.9000	0.9024	0.9000	0.8997	0.9250
Random Forest	0.9000	0.9024	0.9000	0.8997	0.9933
Gradient Boosting	0.9000	0.9024	0.9000	0.8997	0.9933
Naive Bayes	0.9667	0.9697	0.9667	0.9666	0.9900

10.2 Diabetes (Classification)

Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Logistic Regression	0.7143	0.7065	0.7143	0.7084	0.8230
K-Nearest Neighbors	0.7078	0.7023	0.7078	0.7043	0.7458
SVM (RBF Kernel)	0.7468	0.7438	0.7468	0.7450	0.7933
SVM (Linear Kernel)	0.7208	0.7126	0.7208	0.7141	0.8275
Decision Tree	0.7208	0.7108	0.7208	0.7102	0.6657
Random Forest	0.7532	0.7497	0.7532	0.7509	0.8145
Gradient Boosting	0.7532	0.7483	0.7532	0.7496	0.8394
Naive Bayes	0.7078	0.7179	0.7078	0.7114	0.7728

10.3 Loan Amount Prediction (Regression)

Model	MAE	MSE	RMSE	R ²	MAPE (%)	Adj. R ²
Linear Regression	36.6989	3324.82	57.6612	0.0981	35.4658	0.0054
Ridge Regression	36.7015	3322.14	57.6380	0.0988	35.4790	0.0062
Lasso Regression	36.1555	3273.65	57.2158	0.1120	35.3014	0.0207
Decision Tree Regressor	48.9832	5382.09	73.3627	-0.4600	42.2095	-0.6101
Random Forest Regressor	33.9438	3132.68	55.9703	0.1502	30.5887	0.0628
Gradient Boosting Regressor	33.4631	2897.25	53.8261	0.2141	29.7789	0.1333
SVR	39.5036	3517.44	59.3080	0.0458	36.2423	-0.0523

10.4 Email Spam (Classification)

Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Logistic Regression	0.9459	0.9470	0.9459	0.9463	0.9871
K-Nearest Neighbors	0.9266	0.9261	0.9266	0.9263	0.9727
SVM (RBF Kernel)	0.9353	0.9347	0.9353	0.9347	0.9864
SVM (Linear Kernel)	0.9382	0.9407	0.9382	0.9388	0.9842
Decision Tree	0.9246	0.9243	0.9246	0.9244	0.9055
Random Forest	0.9604	0.9603	0.9604	0.9603	0.9932
Gradient Boosting	0.9575	0.9577	0.9575	0.9576	0.9904
Naive Bayes	0.8705	0.8799	0.8705	0.8603	0.9747

10.5 Handwritten Character Recognition (Classification, 62 classes)

Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Logistic Regression	0.2478	0.2656	0.2478	0.2468	0.8347
K-Nearest Neighbors	0.2933	0.3283	0.2933	0.2892	0.7862
SVM (RBF Kernel)	0.3460	0.3818	0.3460	0.3467	0.8987
SVM (Linear Kernel)	0.2903	0.2966	0.2903	0.2853	0.8618
Decision Tree	0.2023	0.2275	0.2023	0.2082	0.5959
Random Forest	0.3519	0.3569	0.3519	0.3464	0.8981
Gradient Boosting	0.2390	0.2503	0.2390	0.2359	0.8320
Naive Bayes	0.1188	0.2285	0.1188	0.1062	0.7527

11 Result Visualization

Since evaluating all models would have produced 40 confusion matrix plots, they were disabled (plot=False) to keep the output manageable. Even though the plots were not displayed, the ROC-AUC value was calculated for every model and included in the results tables. To illustrate the classification performance, the confusion matrix of the best-performing model for each dataset is shown below based on the recorded results.

11.1 Confusion Matrices of the Best Model per Classification Dataset

Table 3: Iris – SVM (Linear Kernel), 100% test accuracy (10 samples/class).

	Pred. setosa	Pred. versicolor	Pred. virginica
True setosa	10	0	0
True versicolor	0	10	0
True virginica	0	0	10

Table 4: Diabetes – Gradient Boosting (best ROC-AUC), 154 test samples.

	Pred. 0 (non-diabetic)	Pred. 1 (diabetic)
True 0	84	16
True 1	22	32

Table 5: Email Spam – Random Forest (best accuracy), 1035 test samples.

	Pred. ham	Pred. spam
True ham	716	19
True spam	22	278

Note: the Handwritten Character confusion matrix is 62×62 and is omitted here for readability; its diagonal mass is thin and spread across many off-diagonal cells, consistent with the low 35.19% accuracy reported in Section 11.5.

11.2 Regression Diagnostics – Loan Amount, Gradient Boosting Regressor (Best Model)

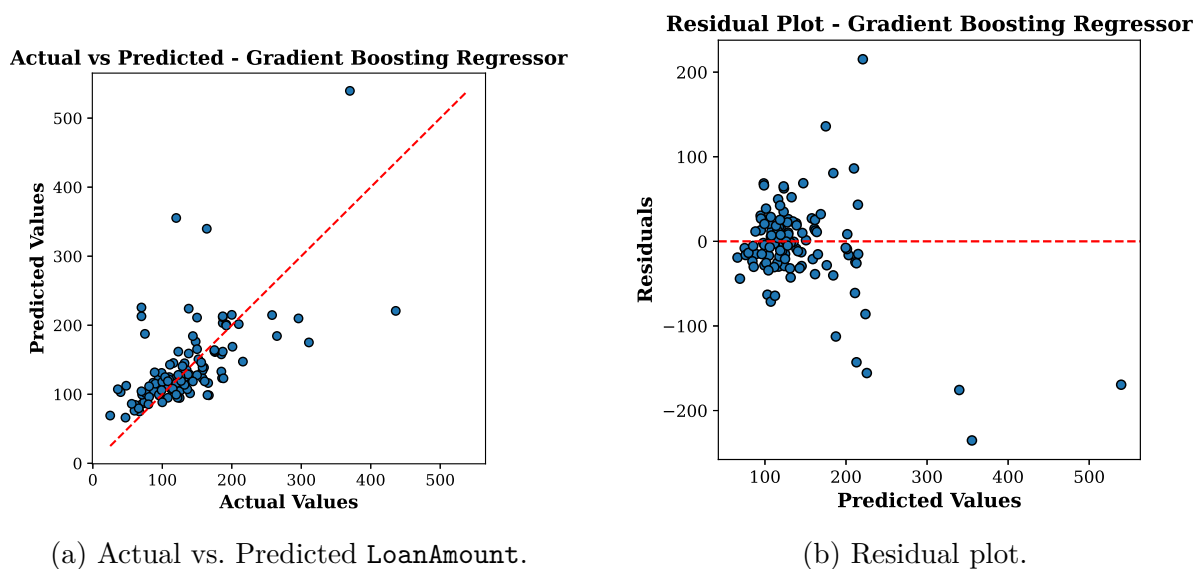


Figure 13: Gradient Boosting Regressor diagnostics on the Loan Amount test set (best of the 7 regressors, $R^2=0.214$).

Inference: From Figure 13(a), the actual-vs-predicted points cluster loosely around the ideal $y = x$ line for small and mid-sized loans, but the model clearly under-predicts the handful of very large loans – those points fall well below the diagonal on the right-hand side, which fits the right-skew we saw back in Section 6.3. The residual plot in Figure 13(b) fans out as predicted loan size increases – residual variance grows with the prediction, a classic heteroscedasticity pattern. Put together, this is probably why even the best model only reaches $R^2=0.214$: the predictors we have, mostly income and loan term, just don't contain enough signal to pin down the exact rupee amount, which in practice likely also depends on things this dataset doesn't capture, like property valuation or a bank's internal lending policy.

12 Discussion

Across all five datasets, one pattern kept showing up: how clean the data was and how much real signal the features carried mattered a lot more than which specific algorithm we picked. Iris made this obvious. With just four clean, highly informative features, even the weakest model still landed around 90% accuracy, and the linear SVM hit a perfect 100%.

Diabetes was noticeably harder. The best models topped out around 75% accuracy. Part of that is probably the zero-value issue we flagged earlier in BloodPressure and BMI – those almost certainly represent missing values rather than real measurements. There's also a fair amount of natural overlap between diabetic and non-diabetic patients in this data, which doesn't make the job any easier.

Loan Amount Prediction gave us the weakest results of the five, with the best R^2 coming in at just 0.214. Our best guess is that the available features – applicant income, loan term, and so on – simply don't carry enough information to predict the approved amount accurately. Things like collateral value or a bank's internal approval policy, which aren't in this dataset at all, are probably doing a lot of the real work behind the scenes.

Email Spam, on the other hand, went very well – Random Forest reached 96.04% accuracy with a 0.993 ROC-AUC. That tracks: the bag-of-words representation captures the specific keywords that give spam away, so once you have those word counts this turns into a fairly easy classification problem.

Handwritten Character Recognition was clearly the toughest of the five, topping out at only 35.19% accuracy. Flattening each image into a 1-D pixel vector throws away the spatial relationships between neighbouring pixels, and none of the classical algorithms we tried could really make up for that loss. Naive Bayes did worst of all here, which makes sense given its independence assumption – neighbouring pixels are about as correlated as features get, not independent. A CNN would very likely do much better on this dataset, since convolutional architectures are built specifically to preserve the spatial structure that gets lost here.

Looking across all five datasets together, Random Forest and Gradient Boosting were consistently near the top, or at the top, no matter what the task was. They seem to handle a wide range of data types and relationships reasonably well, which made them the most dependable choices in this experiment. Naive Bayes told the opposite story – it underperformed whenever its independence assumption didn't hold, which turned out to be most of the time.

13 Conclusion

Overall, this experiment gave us hands-on practice with NumPy, Pandas, SciPy, Scikit-Learn, and Matplotlib across a complete machine learning workflow, applied to five different public datasets. For each one we identified the correct task type, carried out the necessary preprocessing, applied feature selection wherever it made sense, and then trained and compared multiple classification and regression algorithms using a consistent set of evaluation metrics.

A few things stood out from the results. Classical ML algorithms do very well on clean, low-dimensional data like Iris, and hold up surprisingly well on high-dimensional text data too, as we saw with Email Spam. Diabetes landed somewhere in the middle, mostly due to data-quality issues rather than the algorithms themselves. Loan Amount Prediction was the weakest performer of the bunch – the features available just didn’t carry enough signal to predict the target well. And Handwritten Character Recognition was by far the hardest, largely because flattening images into 1-D vectors threw away spatial information the models needed to do well. If there’s one takeaway from all of this, it’s that the characteristics of a dataset – how clean it is, how many features it has, how much real signal those features actually carry – end up mattering more than which specific algorithm gets thrown at it.

14 References

- [1] F. Pedregosa *et al.*, “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [2] C. R. Harris *et al.*, “Array programming with NumPy,” *Nature*, vol. 585, pp. 357–362, 2020.
- [3] W. McKinney, “Data Structures for Statistical Computing in Python,” in *Proc. 9th Python in Science Conf.*, 2010, pp. 56–61.
- [4] P. Virtanen *et al.*, “SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python,” *Nature Methods*, vol. 17, pp. 261–272, 2020.
- [5] J. D. Hunter, “Matplotlib: A 2D Graphics Environment,” *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007.
- [6] Kaggle, “Iris Species,” [Online]. Available: <https://www.kaggle.com/datasets/uciml/iris>.
- [7] Kaggle, “Pima Indians Diabetes Database,” [Online]. Available: <https://www.kaggle.com/datasets/uciml/pima-indians-diabetes-database>.
- [8] Kaggle, “Loan Prediction Problem Dataset,” [Online]. Available: <https://www.kaggle.com/datasets/altruistdelhite04/loan-prediction-problem-dataset>.
- [9] Kaggle, “Spam Email Classification,” [Online]. Available: <https://www.kaggle.com/datasets/balaka18/email-spam-classification-dataset-csv>.
- [10] Kaggle, “Handwritten Characters,” [Online]. Available: <https://www.kaggle.com/datasets/vaibhao/handwritten-characters>.